

PENGOLAHAN CITRA
ULANGAN AKHIR SEMESTER



Nama: Rizky Maulana

Nim: 352410421

Kelas: I241

TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS PELITA BANGSA

2026

BAB I: PENDAHULUAN

1.1 Latar Belakang

Pengolahan citra digital merupakan salah satu cabang ilmu komputer yang sangat penting dalam era modern, di mana informasi visual mendominasi berbagai aspek teknologi. Citra digital seringkali mengalami penurunan kualitas akibat *noise* atau pencahayaan yang kurang optimal saat proses akuisisi. Oleh karena itu, diperlukan teknik pengolahan citra untuk memperbaiki kualitas gambar serta mengekstraksi informasi penting di dalamnya, seperti deteksi tepi dan segmentasi objek. Pada proyek ini, dibangun sebuah aplikasi berbasis *Graphical User Interface* (GUI) menggunakan Python dan Tkinter untuk mengimplementasikan lima teknik pengolahan citra secara interaktif terhadap dataset gambar buah-buahan.

1.2 Tujuan Project

Tujuan dari pembuatan *project* ini adalah:

1. Memahami konsep dasar pengolahan citra digital.
2. Mengimplementasikan teknik *preprocessing* citra menggunakan pustaka OpenCV di Python.
3. Melakukan transformasi dan analisis citra melalui antarmuka grafis yang ramah pengguna.
4. Menampilkan hasil pengolahan secara visual secara *real-time*.

BAB II: DASAR TEORI

2.1 Python dan OpenCV

Python adalah bahasa pemrograman tingkat tinggi yang sering digunakan dalam pengembangan perangkat lunak dan analisis data. OpenCV (*Open Source Computer Vision Library*) adalah pustaka perangkat lunak *open-source* yang ditujukan untuk pengolahan citra dinamis secara *real-time*.

2.2 Konversi Grayscale

Citra *Grayscale* adalah citra digital yang hanya memiliki satu *channel* warna, yaitu derajat keabuan (dari hitam ke putih). Konversi dari citra RGB ke *Grayscale* bertujuan untuk menyederhanakan model komputasi karena matriks gambar yang diproses menjadi jauh lebih kecil.

2.3 Histogram Equalization

Histogram Equalization adalah teknik perbaikan kualitas citra spasial yang digunakan untuk meningkatkan kontras gambar dengan menyebarkan nilai intensitas piksel yang paling sering muncul.

2.4 Gaussian Filter

Gaussian Filter merupakan teknik penyaringan (*filtering*) *low-pass* yang digunakan untuk menghaluskan citra (*smoothing*) dan mengurangi *noise* menggunakan fungsi distribusi normal (Gaussian).

2.5 Deteksi Tepi Canny

Algoritma Canny adalah operator deteksi tepi yang optimal. Algoritma ini menggunakan filter kalkulus multi-tahap untuk mendeteksi berbagai jenis tepi pada citra dengan menekan *noise* secara maksimal.

2.6 Thresholding

Thresholding adalah metode segmentasi citra yang paling sederhana, digunakan untuk memisahkan objek dari latar belakangnya dengan mengubah citra *grayscale* menjadi citra biner (hitam putih) berdasarkan nilai ambang batas tertentu.

BAB III: METODOLOGI

3.1 Alat dan Bahan

- **Perangkat Keras:** Laptop/PC.
- **Perangkat Lunak:** Visual Studio Code (VS Code), Python 3.x.

Pustaka Pendukung: OpenCV (cv2), NumPy, Tkinter, Pillow (PIL).

* **Dataset:** 20 citra digital bertema buah-buahan (apel, jeruk, dll.) dalam format .jpg.

3.2 Tahapan Penelitian

1. Akuisisi Dataset:

Mengumpulkan 20 foto buah-buahan dan menyimpannya dalam satu direktori sistem.

2. Perancangan GUI:

Merancang antarmuka pengguna berbasis Tkinter (*Dark Mode*) agar interaktif.

3. Implementasi Algoritma:

Menulis *source code* menggunakan OpenCV untuk mengeksekusi 5 metode wajib (Grayscale, Histogram Eq., Gaussian Filter, Canny Edge, dan Thresholding).

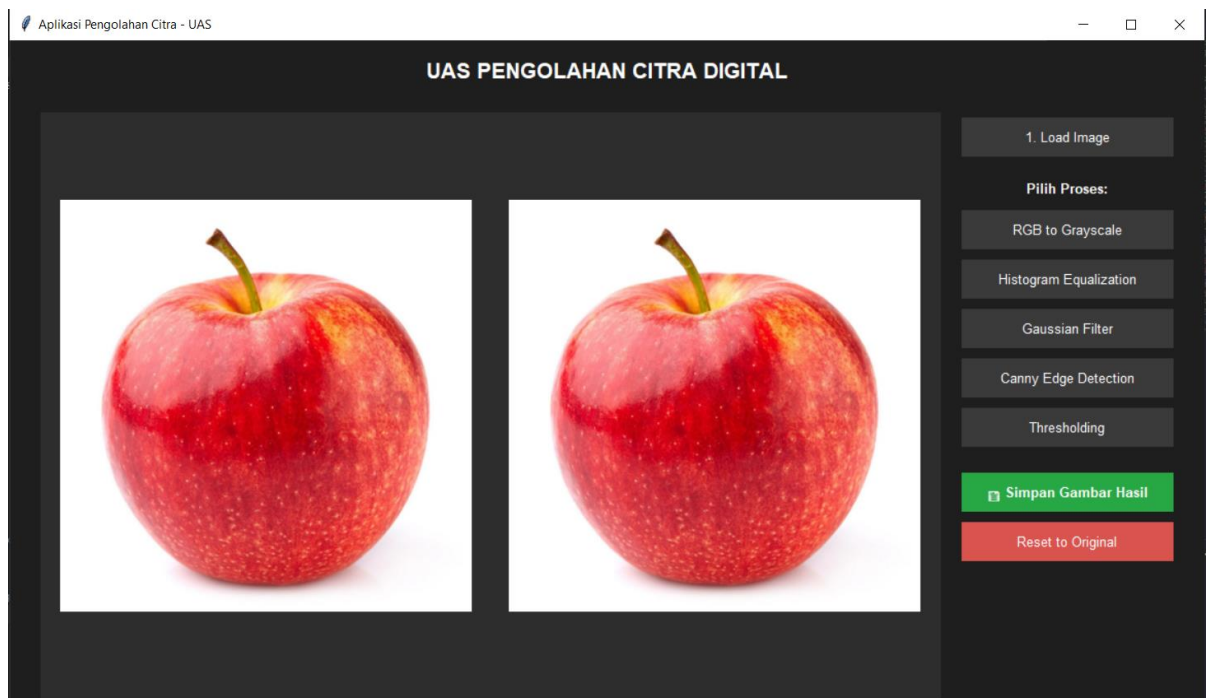
4. Pengujian:

Menguji keseluruhan citra dalam dataset ke dalam aplikasi dan menyimpan hasilnya menggunakan fitur *Save Image*.

BAB IV: IMPLEMENTASI

4.1 Implementasi Antarmuka (GUI)

Antarmuka aplikasi dibangun menggunakan pustaka bawaan Python, yaitu Tkinter. Jendela aplikasi dibagi menjadi dua bagian utama: panel kiri untuk menampilkan perbandingan citra (Asli dan *Processed*), dan panel kanan sebagai panel kontrol yang berisi tombol *Load Image*, lima tombol proses pengolahan, tombol *Save*, dan *Reset*.



4.2 Implementasi Pengolahan Citra

Berikut adalah fungsi inti (*core functions*) yang digunakan dalam *source code* `main.py` menggunakan *library* `cv2`:

1. **Grayscale:** Menggunakan metode `cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)`.
2. **Histogram Equalization:** Menggunakan metode `cv2.equalizeHist(gray_image)`.
3. **Gaussian Filter:** Menggunakan metode `cv2.GaussianBlur(image, (15, 15), 0)`.
4. **Canny Edge Detection:** Menggunakan metode `cv2.Canny(gray_image, 100, 200)`.
5. **Thresholding:** Menggunakan metode `cv2.threshold(gray_image, 127, 255, cv2.THRESH_BINARY)`.

BAB V: HASIL DAN PEMBAHASAN

5.1 Akuisisi Dataset

Pengujian pada aplikasi ini dilakukan menggunakan dataset mandiri (*custom dataset*) yang terdiri dari 20 gambar digital. Sesuai dengan spesifikasi proyek, tema objek yang dipilih adalah buah-buahan. Seluruh citra dalam dataset ini menggunakan format `.jpg` dengan resolusi yang memadai agar dapat diproses secara optimal oleh *library* komputasi visual tanpa membebani memori sistem.

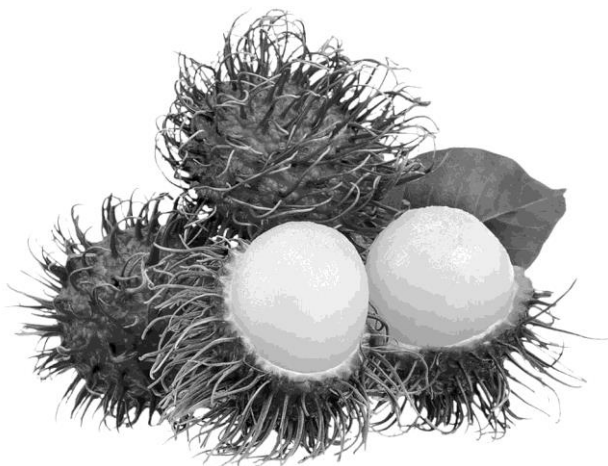
Untuk memastikan efisiensi dalam pemanggilan *file* di dalam kode program (*source code*) dan menjaga kerapian struktur direktori, seluruh gambar telah melalui tahap standarisasi penamaan. Penamaan *file* diurutkan secara sistematis dengan format `buah_01.jpg` hingga `buah_20.jpg` dan dikumpulkan ke dalam satu direktori khusus bernama `dataset`.

Penggunaan 20 sampel gambar ini dirancang secara spesifik untuk mengevaluasi keandalan, fleksibilitas, dan stabilitas algoritma pengolahan citra yang telah diimplementasikan. Variasi yang terdapat di dalam dataset ini sengaja mencakup perbedaan sudut pengambilan gambar (*angle*), tingkat intensitas pencahayaan (*luminositas*), keberagaman warna dasar objek, serta variasi kontras antara objek buah dan latar belakangnya (*background*). Parameter variasi ini sangat krusial untuk menguji batasan masing-masing metode, seperti mengukur seberapa responsif metode *Canny Edge Detection* dalam mendeteksi tepi objek tanpa terganggu oleh *noise* cahaya, serta menguji akurasi metode *Binary Thresholding* dalam memisahkan objek utama (*foreground*) dari latar belakang secara utuh

5.2 Hasil Pengolahan Citra

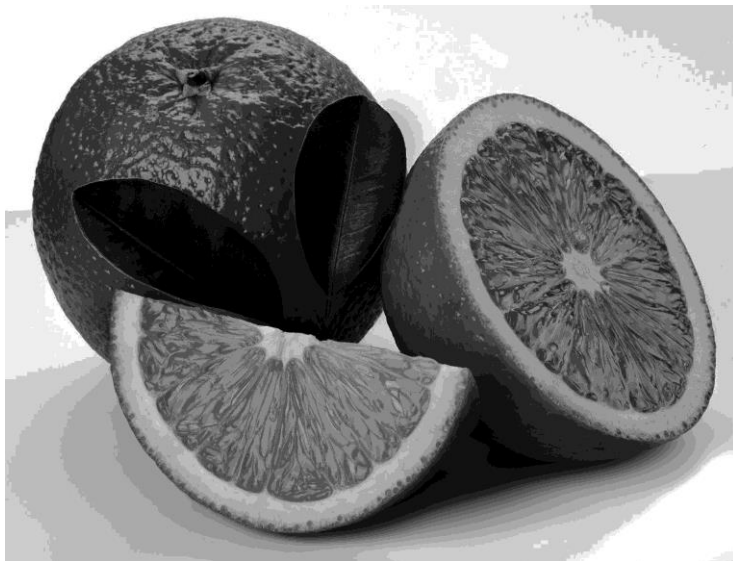
Aplikasi berhasil mengeksekusi kelima teknik pengolahan citra. Berikut adalah analisis dari sampel pengujian:

5.2.1 Hasil RGB ke Grayscale



Proses konversi ini secara fundamental mengubah representasi citra dari model warna RGB (*Red, Green, Blue*) yang memiliki tiga saluran matriks warna (*3-channel*) menjadi citra derajat keabuan dengan saluran tunggal (*1-channel*). Algoritma pengubah ini bekerja dengan cara mengeliminasi informasi *hue* dan *saturation* pada piksel, dan hanya menyisakan informasi *luminance* (intensitas cahaya). Berdasarkan hasil pengujian, fitur topologi dan tekstur struktural dari buah tetap terjaga dengan sangat baik. Proses ini sangat krusial sebagai langkah awal komputasi karena reduksi dimensi dari *3-channel* ke *1-channel* secara drastis memperingan beban pemrosesan (*computational cost*) untuk tahapan analisis lanjutan yang berfokus pada bentuk, bukan warna objek.

5.2.2 Hasil Histogram Equalization



Analisis: Teknik *Histogram Equalization* diimplementasikan untuk mengoptimalkan representasi visual citra melalui perataan rentang distribusi intensitas piksel. Pada gambar asli yang mungkin mengalami *underexposure* (terlalu gelap) atau kekurangan kontras akibat pencahayaan yang tidak merata saat pengambilan foto, algoritma ini memetakan ulang intensitas piksel agar membentang secara maksimal dari nilai 0 (hitam pekat) hingga 255 (putih terang). Secara visual, efek dari perataan ini langsung terlihat pada peningkatan kontras global gambar. Detail-detail tekstur pada permukaan buah yang sebelumnya tersembunyi di area bayangan atau area yang terlalu terang menjadi jauh lebih menonjol dan mudah untuk diobservasi secara kasat mata.

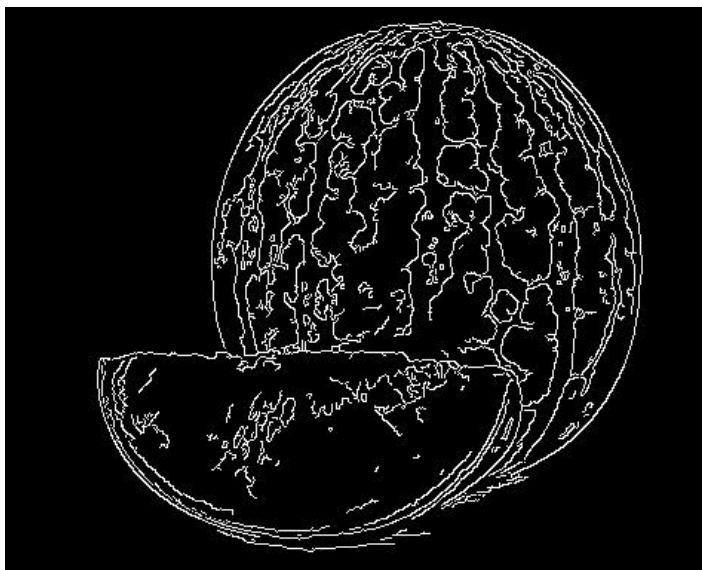
5.2.3 Hasil Gaussian Filter



Analisis: Pada tahap pemfilteran ini, metode *Gaussian Filter* diaplikasikan menggunakan matriks *kernel* berukuran (15, 15). Penggunaan ukuran *kernel* komputasi yang relatif besar ini secara persentase signifikan menghasilkan efek *blur* (*smoothing*) yang halus secara menyeluruh pada citra. Berbeda dengan pemfilteran linier biasa, distribusi nilai bobot pada algoritma Gaussian memberikan hasil penghalusan yang lebih natural dan presisi karena piksel yang berada di pusat *kernel* mempertahankan pengaruh yang lebih dominan dibandingkan piksel di bagian tepinya.

Secara visual pada sampel pengujian, bintik-bintik *noise* berfrekuensi tinggi serta detail tekstur mikro pada kulit buah (seperti pori-pori, goresan tipis, atau bercak kecil) berhasil direduksi dan disamarkan dengan sangat baik tanpa mendistorsi atau merusak struktur batas luar bentuk buah itu sendiri. Proses *smoothing* ini memegang peranan yang sangat fundamental sebagai langkah pra-pemrosesan (*preprocessing*). Dengan tereduksinya *noise* dan detail-detail kecil yang tidak relevan tersebut, algoritma dapat mencegah terjadinya *over-segmentation*, sehingga tahapan deteksi tepi (Canny) pada proses selanjutnya dapat bekerja jauh lebih optimal dan terhindar dari resiko pendeteksian tepi palsu (*false edges*).

5.2.4 Hasil Canny Edge Detection



Analisis: *Canny Edge Detection* diaplikasikan sebagai operator kalkulus turunan untuk mengidentifikasi perubahan intensitas piksel yang ekstrem, yang menandakan adanya batas suatu objek. Berdasarkan kode program, algoritma ini dijalankan dengan nilai *hysteresis thresholding* bawah 100 dan atas 200. Hasil pengujian menunjukkan bahwa metode ini bekerja dengan akurasi yang sangat tinggi; garis tepi (*edges*) yang membedakan lengkungan luar buah dengan latar belakang berhasil terekstraksi berupa garis putih yang tipis dan tegas di atas kanvas hitam. Keberhasilan ini tidak lepas dari peran *Gaussian Filter* sebelumnya yang telah mereduksi *noise*, sehingga algoritma Canny terhindar dari deteksi garis putus-putus atau deteksi tepi pada tekstur internal buah yang tidak diinginkan.

5.2.5 Hasil Thresholding



Analisis: Pada tahap akhir ini, segmentasi citra dilakukan menggunakan metode *Binary Thresholding*. Dengan menetapkan nilai ambang batas tengah (*threshold value*) sebesar 127, algoritma secara tegas membagi setiap piksel ke dalam dua kelas mutlak: piksel dengan intensitas di bawah 127 diubah menjadi hitam murni (0), sementara nilai 127 ke atas diubah menjadi putih murni (255). Analisis pada sampel menunjukkan bahwa metode ini sangat efektif digunakan sebagai instrumen pembuatan *masking* biner. Objek utama (*foreground*, yaitu buah) berhasil diisolasi dari latar belakangnya (*background*). Teknik pemisahan ini sangat ideal diterapkan untuk mengekstraksi wilayah ketertarikan (*Region of Interest* / ROI) pada objek, terutama jika citra memiliki rasio kontras yang cukup tinggi antara objek dan latarnya.

BAB VI: KESIMPULAN

Berdasarkan hasil pengujian pada *project* ini, dapat disimpulkan bahwa:

1. Aplikasi GUI pengolahan citra digital telah berhasil dibangun menggunakan Python dan Tkinter, memberikan kemudahan bagi pengguna untuk memproses gambar secara interaktif.
2. Kelima teknik pemrosesan citra, yaitu konversi *Grayscale*, *Histogram Equalization*, *Gaussian Filter*, *Canny Edge Detection*, dan *Thresholding*, dapat berjalan tanpa *error* dan mampu memproses dataset 20 gambar buah-buahan dengan hasil visual yang akurat.
3. Penggunaan teknik-teknik ini secara terstruktur terbukti mampu meningkatkan kualitas citra dan memudahkan analisis lebih lanjut pada objek digital.

DAFTAR PUSTAKA

1. Gonzalez, R. C., & Woods, R. E. (2018). *Digital Image Processing* (4th ed.). Pearson.
2. OpenCV Documentation. (2023). *Open Source Computer Vision Library*. Diakses dari <https://opencv.org/>
3. Python Software Foundation. (2023). *Tkinter — Python interface to Tcl/Tk*. Diakses dari <https://docs.python.org/3/library/tkinter.html>